

T-302-HONN: Lab 7

Guðmundur Alexander Magnússon
gudmundurm24@ru.is

Hafþór Haugen
hafthor24@ru.is

10. október 2026

Tölvunarfræðideild

1) Docker setup (10 points)

1. Output after: docker pull python

```
Using default tag: latest
latest: Pulling from library/python
58c447da347e: Pulling fs layer
58c447da347e: Pull complete
6eefb2f5d3e9: Pull complete
42f0cc32f2e3: Pull complete
7654fbfc185d: Pull complete
e7272a918094: Pull complete
e636a39670e0: Pull complete
fd25b15610c9: Download complete
8bedc2311dd5: Download complete
Digest: sha256:504679006065cdab9a85e983e269140d3c5a279492395a6863b25b1ffb377f86
Status: Downloaded newer image for python:latest
docker.io/library/python:latest
```

2. The command downloads the python image, defaulting to the latest one from the Docker Hub registry and stores it locally as a stack of cached layers, so containers can be created without downloading again.

2 Container Theory (40 points)

1. (Total point: 10) What is a container and why do we want to use them when developing software and in production?

A container is a standardized unit of software that allows us to easily package code and its dependencies, easily set up and run code anywhere and containerize the program we are running. Containers do not contain their own full operating systems, instead they share an OS kernel to save resources and make them more lightweight. Despite sharing an OS they are still isolated.

Containers also solve a myriad of problems such as: “works on my machine”, slow deployment, scaling applications and reducing the resource load compared to VMs.

2. (Total point: 10) What is a Container Image?

To make a container you first need to create a container image. The image is a blueprint or template with all the information needed to run the application, such as dependencies, configuration, scripts and binaries.

Images are made up of multiple independent layers and can be based on other images e.g. a python image which has everything needed to run python already set up.

3. (Total point: 10) What is a container orchestration tool and what purpose does it serve?

It´s a tool that manages containerized workloads and services across a cluster of machines. Kubernetes (K8s) is a good example, its a portable extensible open-source platform, it provides a framework for running distributed systems resiliently. Docker runs on one machine. Docker compose starts a multi-container app from one YAML file, but it's a development tool, it gets awkward in production once you need multiple host machines, Kubernetes is used instead for those needs. Compose's drawback makes the case directly, scaling is harder, managing availability is harder, performance suffers under traffic spikes, backups and security get harder. An orchestration tool is what you move to when the system overgrows a single host. What it does: Scaling - horizontal scaling, which spins up extra instances as load rises, releases resources as load falls. Failover and self-healing - restarts and replaces containers when they fail. Load balancing - distributes network traffic across application containers. And service discovery - containers can talk to each other inside the cluster by DNS name, to name a few. With the Kubernetes container orchestration tool it's easy to migrate from one cloud provider to another, or to on-premises servers.

4. (Total point: 10)What is the problem with this Dockerfile? Explain

```
# syntax=docker/dockerfile:1
FROM python:3.8-slim-buster

WORKDIR /app

COPY . .

RUN pip install -r requirements.txt

CMD ["python", "main.py"]
```

The problem is the order of the Copy commands, the current order copies all the application files before the dependencies are installed. If any of the application files change, Docker cannot reuse the cached layers after COPY . ., so it has to rerun the RUN pip install -r requirements.txt command. A better order would be to COPY requirements.txt, install the requirements.txt and then COPY . . to get the rest of the application. The improved order is better for Docker layer caching and hence builds faster.